

INSTRUCTION SET of 8086

Continued

Arithmetic Instructions

- **Arithmetic instructions include**
 - **addition** **ADD, ADC (AAA, DAA)**
 - **subtraction** **SUB, SBB (AAS, DAS)**
 - **multiplication** **MUL, IMUL (AAM)**
 - **division** **DIV, IDIV (AAD,CBW,CWD)**
 - **negation** **NEG**
 - **increment** **INC**
 - **Decrement** **DEC**

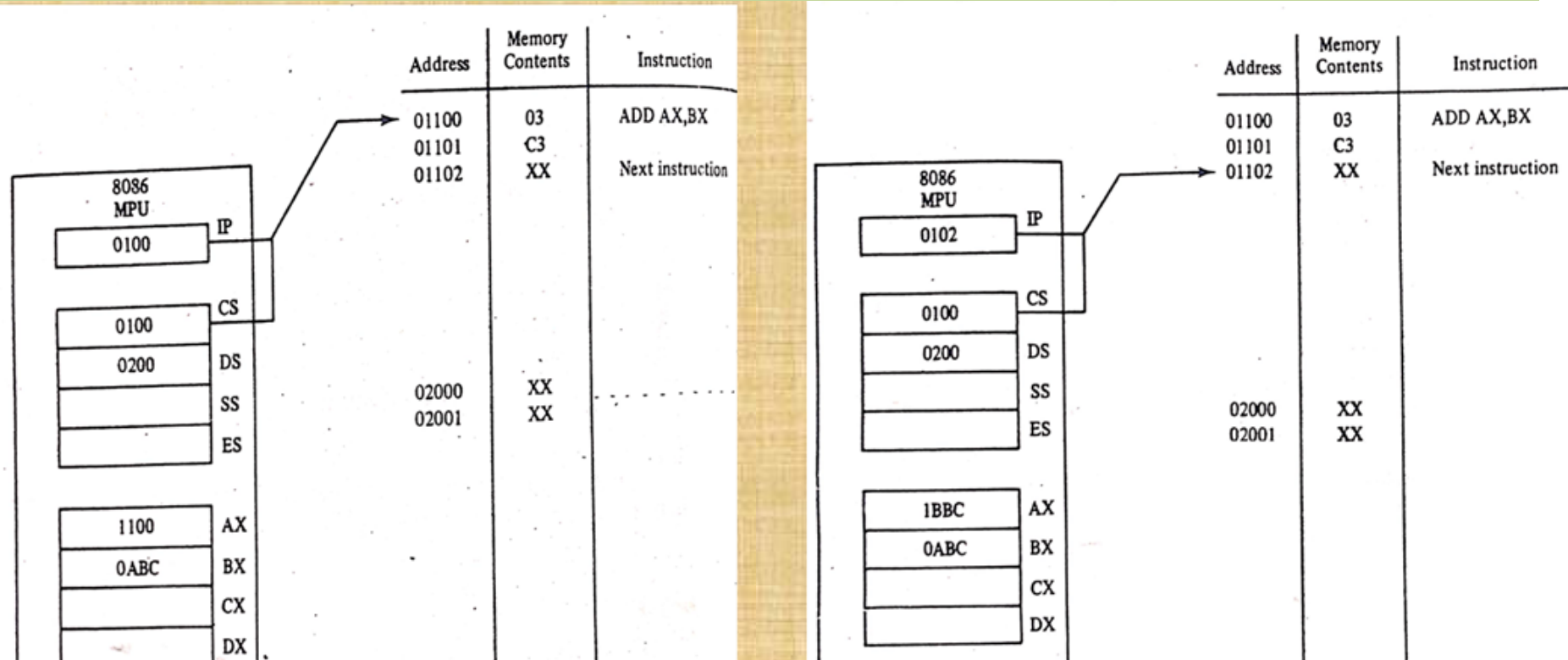
Addition Instructions (ADD)

Mnemonic	Meaning	Format	Operation	Flags Affected
ADD	Addition	ADD D, S	(S) + (D) → (D) Carry → (CF)	OF, SF, ZF, AF, PF, CF

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Assembly Language	Operation
ADD AL,BL	AL = AL + BL
ADD CX,DI	CX = CX + DI
ADD CL, 044H	CL = CL + 44H
ADD BX, 0245FH	BX = BX + 245FH
ADD [BX],AL	AL adds to the contents of the data segment memory location address by BX with the sum stored in the same memory location
ADD CL,[BP]	The byte contents of the stack segment memory location addressed by BP add to CL with the sum stored in CL
ADD BX,[SI + 2]	The word contents of the data segment memory location addressed by the sum of SI plus 2 add to BX with the sum stored in BX
ADD CL,TEMP	The byte contents of the data segment memory location TEMP add to CL with the sum stored in CL
ADD BX,TEMP[DI]	The word contents of the data segment memory location addressed by TEMP plus DI add to BX with the sum stored in BX

Addition Instructions (ADD)



```

0000  B0 00          MOV    AL,0           ;clear sum
0002  BE 0003        MOV    SI,3          ;address element 3
0005  02 84 0002 R   ADD    AL,ARRAY[SI]   ;add element 3
0009  02 84 0004 R   ADD    AL,ARRAY[SI+2] ;add element 5
000D  02 84 0006 R   ADD    AL,ARRAY[SI+4] ;add element 7
0011  C3            RET
    
```


Addition Instructions (ADC)

Mnemonic	Meaning	Format	Operation	Flags Affected
ADC	Add with carry	ADC D, S	$(S) + (D) + (CF) \rightarrow (D)$ Carry $\rightarrow (CF)$	OF, SF, ZF, AF, PF, CF

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Assembly Language

Operation

ADC AL,AH

$AL = AL + AH + \text{carry}$

ADC CX,BX

$CX = CX + BX + \text{carry}$

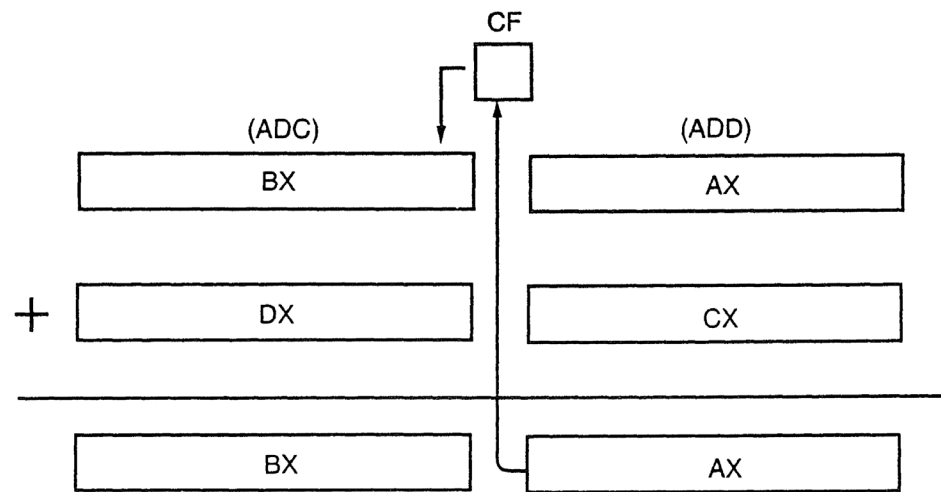
ADC BX,[BP + 2]

The word contents of the stack segment memory location address by BP plus 2 add to BX with carry with the sum stored in BX

Addition Instructions (ADC)

Ex: Add Two 32bits binary numbers that reside in (BX,AX) and (DX,CX), store the result in (BX,AX)

FIGURE Additional-with-carry showing how the carry flag (C) links the two 16-bit additions into one 32-bit addition



0000	03	C1	ADD	AX, CX
0002	13	DA	ADC	BX, DX
0004	C3		RET	

Addition Instructions (INC)

Mnemonic	Meaning	Format	Operation	Flags Affected
INC	Increment by 1	INC D	$(D) + 1 \rightarrow (D)$	OF, SF, ZF, AF, PF

Destination

Reg16

Reg8

Memory

<i>Assembly Language</i>	<i>Operation</i>
INC BL	$BL = BL + 1$
INC SP	$SP = SP + 1$
INC DATA1	Increments the contents of data segment memory location DAAT1

Addition Instructions (AAA)

Mnemonic	Meaning	Format	Operation	Flags Affected
AAA	ASCII adjust for addition	AAA		AF, CF OF, SF, ZF, PF undefined

ASCII Adjust after Addition.

Corrects result in AH and AL after addition when working with BCD values.

It works according to the following Algorithm:

if low nibble of AL > 9 or AF = 1 then:

- AL = AL + 6
- AH = AH + 1
- AF = 1
- CF = 1

else

- AF = 0
- CF = 0

in both cases:

clear the high nibble of AL.

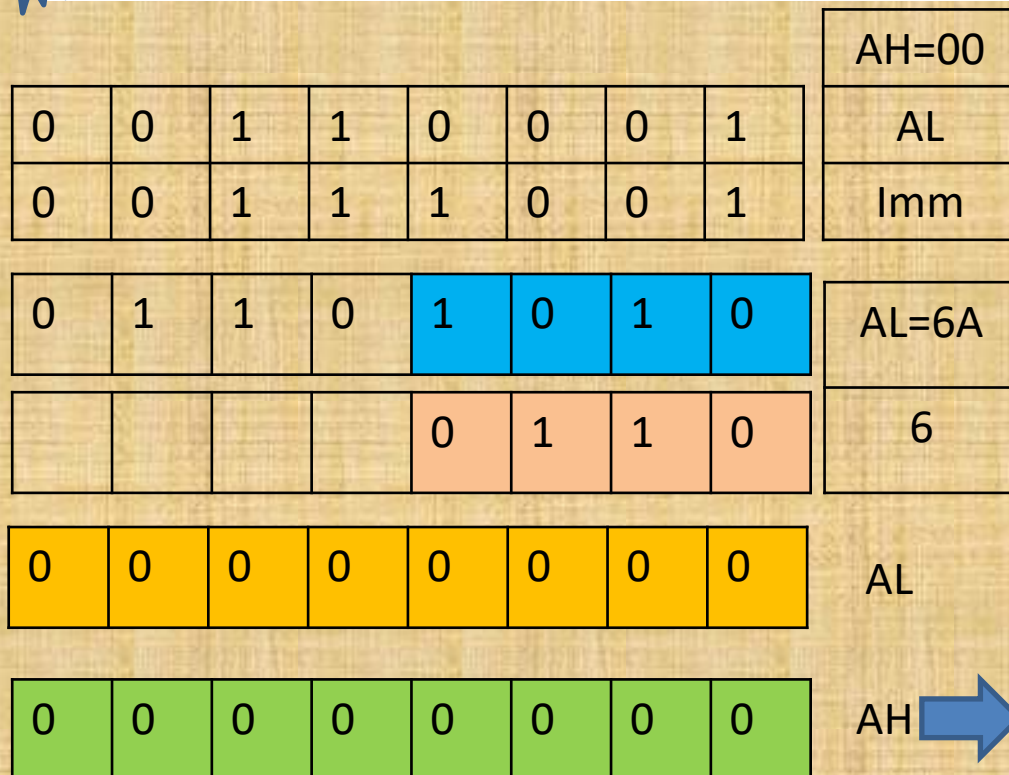
Addition Instructions (AAA)

```
MOV    AX,031H    ;load ASCII 1
ADD    AL,039H     ;add ASCII 9
AAA                     ;adjust
ADD    AX,03030H   ;answer to ASCII
```

31+39 =6A Hex or

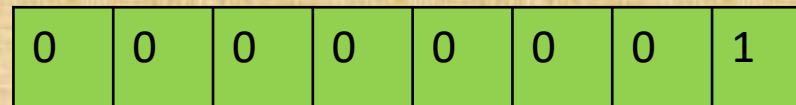
1+9=10 decimal

31 + 39 = 3130 ASCII



ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL



Addition Instructions (DAA)

Mnemonic	Meaning	Format	Operation	Flags Affected
DAA	Decimal adjust for addition	DAA		SF, ZF, AF, PF, CF, OF, undefined

Decimal adjust After Addition.

Corrects the result of addition of two packed BCD values.

Algorithm:

```
if low nibble of AL > 9 or AF = 1 then:
```

- $AL = AL + 6$
- $AF = 1$

```
if  $AL > 9Fh$  or  $CF = 1$  then:
```

- $AL = AL + 60h$
- $CF = 1$

MOV	DX,01234H	;load 1,234
MOV	BX,03099H	;load 3,099
MOV	AL,BL	;sum BL with DL
ADD	AL,DL	
DAA		;adjust
MOV	CL,AL	;answer to CL
MOV	AL,BH	;sum BH, DH, and carry
ADC	AL,DH	
DAA		;adjust
MOV	CH,AL	;answer to CH

Decimal adjust After Addition.

Corrects the result of addition of two packed BCD values.

Algorithm:

if low nibble of AL > 9 or AF = 1 then:

- AL = AL + 6
- AF = 1

if AL > 9Fh or CF = 1 then:

- AL = AL + 60h
- CF = 1

DH	DL	BH	BL	AL
12	34	30	99	99

MOV DX,.. MOV BX,.. MOV AL,BL

ADD AL,DL

AL
C D

1001 1001
0011 0100

1100 1101
C D

LOW D > 9

AL
± 6

1100 1101
0000 0110

1101 0011
D 3

D379F

AL	CL
33	33

Copy

AL
30

ADC AL,DH

0011 0000
0001 0010

0100 0011

CF ← 1

3 > 9 AF = 1

AL = 43 CF = 1

No Operation

Result

AL	CH	CL
43	43	33

Subtraction Instructions (SUB)

Mnemonic	Meaning	Format	Operation	Flags affected
SUB	Subtract	SUB D,S	(D) – (S) → (D) Borrow → (CF)	OF, SF, ZF, AF, PF, CF

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Assembly Language

Operation

SUB CL,BL	CL = CL – BL
SUB AX,SP	AX = AX – SP
SUB DH, 06FH	DH = DH – 6FH
SUB AX,0CCCCH	AX = AX – CCCCH
SUB [DI],CH	Subtracts the contents of CH from the contents of the data segment memory location addressed by DI
SUB CH,[BP]	Subtracts the byte contents of the stack segment memory location address by BP from CH
SUB AH,TEMP	Subtracts the byte contents of the data segment memory location TEMP from AH

Subtraction Instructions (SUB)

Ex: Calculate the content of CH and the flags (S, Z, A, C, P, O) after executing the following code

```
0000 B5 22      MOV  CH,022H
0002 80 ED 44    SUB  CH,044H
```

Z = 0 (result not zero)
C = 1 (borrow)
A = 1 (half-borrow)
S = 1 (result negative)
P = 1 (even parity)
O = 0 (no overflow)

0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0

1	1	1	0	1	1	1	1	0
---	---	---	---	---	---	---	---	---

Subtraction Instructions (SBB)

Mnemonic	Meaning	Format	Operation	Flags affected
SBB	Subtract with borrow	SBB D,S	$(D) - (S) - (CF) \rightarrow (D)$	OF, SF, ZF, AF, PF, CF

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Assembly Language

Operation

SBB AH,AL	$AH = AH - AL - \text{carry}$
SBB AX,BX	$AX = AX - BX - \text{carry}$
SBB CL, 2	$CL = CL - 2 - \text{carry}$
SBB [DI],AL	Both AL and carry subtract from the data segment memory location addressed by DI
SBB DI,[BP + 2]	Both carry and the word contents of the stack segment memory location addressed by the sum of BP and 2 subtract from DI

Subtraction Instructions (DEC)

Mnemonic	Meaning	Format	Operation	Flags affected
DEC	Decrement by 1	DEC D	$(D) - 1 \rightarrow (D)$	OF, SF, ZF, AF, PF

Destination

Reg16

Reg8

Memory

<i>Assembly Language</i>	<i>Operation</i>
DEC BH	$BH = BH - 1$
DEC CX	$CX = CX - 1$
DEC NUMB	Subtracts 1 from the contents of the data segment memory location NUMB

Subtraction Instructions (NEG)

Mnemonic	Meaning	Format	Operation	Flags affected
NEG	Negate	NEG D	$0 - (D) \rightarrow (D)$ $1 \rightarrow (CF)$	OF, SF, ZF, AF, PF, CF

Destination
Reg Memory

Negate. Makes operand negative (two's complement).

Algorithm:

- Invert all bits of the operand
- Add 1 to inverted operand

Example:

```
MOV AL, 5      ; AL = 05h
NEG AL         ; AL = 0FBh (-5)
NEG AL         ; AL = 05h (5)
```

$\begin{array}{r} 0000\ 0101 \\ \hline 1111\ 1010 \\ + \\ \hline 1111\ 1011 \end{array}$	05 ↓ NEG FBh	$\begin{array}{r} 0000 \leftarrow 0 \\ 1111 \leftarrow -1 \\ 1110 \leftarrow -2 \\ 1101 \leftarrow -3 \\ 1100 \leftarrow -4 \\ 1011 \leftarrow -5 \\ 1010 \leftarrow -6 \\ \vdots \\ \vdots \end{array}$
$\begin{array}{r} 1111\ 1011 \\ \hline 0000\ 0100 \\ + \\ \hline 0000\ 0101 \end{array}$	FB ↓ NEG 05	



Subtraction Instructions (DAS)

Mnemonic	Meaning	Format	Operation	Flags affected
DAS	Decimal adjust for subtraction	DAS		SF, ZF, AF, PF, CF OF undefined

Decimal adjust After Subtraction.
Corrects the result of subtraction of two packed BCD values.

Algorithm:

```
if low nibble of AL > 9 or AF = 1 then:
```

- $AL = AL - 6$
- $AF = 1$

```
if  $AL > 9Fh$  or  $CF = 1$  then:
```

- $AL = AL - 60h$
- $CF = 1$

Subtraction Instructions (DAS)

WORK OUT by YOURSELF check the results with emulator

0000	BA 1234	MOV	DX,01234H	;load 1,234
0003	BB 3099	MOV	BX,03099H	;load 3,099
0006	8A C3	MOV	AL,BL	;subtract DL from BL
0008	2A C2	SUB	AL,DL	
000A	2F	DAS		;adjust
000B	8A C8	MOV	CL,AL	;answer to CL
000D	8A C7	MOV	AL,BH	;subtract DH
000F	1A C6	SBB	AL,DH	
0011	2F	DAS		;adjust
0012	8A E8	MOV	CH,AL	;answer to CH

Subtraction Instructions (AAS)

Mnemonic	Meaning	Format	Operation	Flags affected
AAS	ASCII adjust for subtraction	AAS		AF, CF OF, SF, ZF, PF undefined

ASCII Adjust after Subtraction.
Corrects result in AH and AL after subtraction
when working with BCD values.

Algorithm:

if low nibble of AL > 9 or AF = 1 then:

- AL = AL - 6
- AH = AH - 1
- AF = 1
- CF = 1

else

- AF = 0
- CF = 0

in both cases:
clear the high nibble of AL.

Example:

```
MOV AX,0D2FFh    ; AH = 02, AL = 0FFh
AAS               ; AH = 01, AL = 09
RET
```


Multiplication Instructions (MUL)

Mnemonic	Meaning	Format	Operation	Flags Affected
MUL	Multiply (unsigned)	MUL S	$(AL) \cdot (S8) \rightarrow (AX)$ $(AX) \cdot (S16) \rightarrow (DX), (AX)$	OF, CF SF, ZF, AF, PF undefined

Source
Reg8
Reg16
Mem8
Mem16

Assembly Language

Operation

MUL CL

AL is multiplied by CL; the unsigned product is in AX

MUL CX

AX is multiplied by CX; the unsigned product is in DX–AX

MUL TEMP

AL is multiplied by the byte contents of the data segment memory location addressed by TEMP; the unsigned product is in AX

Multiplication Instructions (MUL)

Suppose that BL and CL each contain two 8-bit unsigned numbers, and these numbers must be multiplied to form a 16-bit product stored in DX.

0000	B3	05	MOV	BL, 05	;load data
0002	B1	0A	MOV	CL, 010	
0004	8A	C1	MOV	AL, CL	;position data
0006	F6	E3	MUL	BL	;multiply
0008	8B	D0	MOV	DX, AX	;position product

Multiplication Instructions (IMUL)

Mnemonic	Meaning	Format	Operation	Flags Affected
IMUL	Integer multiply (signed)	IMUL S	$(AL) \cdot (S8) \rightarrow (AX)$ $(AX) \cdot (S16) \rightarrow (DX).(AX)$	OF, CF SF, ZF, AF, PF undefined

Source
Reg8
Reg16
Mem8
Mem16

Assembly Language

Operation

IMUL DI	AX is multiplied by DI; the signed product is in DX–AX
IMUL DH	AL is multiplied by DH; the signed product is in AX

Multiplication Instructions (AAM)

Mnemonic	Meaning	Format	Operation	Flags Affected
AAM	Adjust AX for multiplication	AAM	$Q((AL)/10) \rightarrow AH$ $R((AL)/10) \rightarrow AL$	SF, ZF, PF OF, AF, CF undefined

ASCII Adjust after Multiplication.
Corrects the result of multiplication of two BCD values.

Algorithm:

- $AH = AL / 10$
- $AL = \text{remainder}$

Example:

```
MOV AL, 15    ; AL = 0Fh
AAM           ; AH = 01, AL = 05
RET
```

```
0000  B0 05          MOV    AL,005          ;load multiplicand
0002  B1 05          MOV    CL,005          ;load multiplier
0004  F6 E1          MUL     CL
0006  D4 0A          AAM                     ;adjust
```

AAM Instruction. The AAM instruction follows the multiplication instruction after multiplying two 1-digit unpacked BCD numbers. The example shows a short program that multiplies 5 times 5. The result after the multiplication is 0019H in the AX register. After adjusting the result with the AAM instruction, AX contains a 0205H. This is an unpacked BCD result of 25. If 3030H adds to 0205H, this becomes an ASCII result of 3235H.

Division Instructions (DIV)

Mnemonic	Meaning	Format	Operation	Flags Affected
DIV	Division (unsigned)	DIV S	(1) $Q((AX)/(S8)) \rightarrow (AL)$ $R((AX)/(S8)) \rightarrow (AH)$ (2) $Q((DX,AX)/(S16)) \rightarrow (AX)$ $R((DX,AX)/(S16)) \rightarrow (DX)$ If Q is FF_{16} in case (1) or $FFFF_{16}$ in case (2), then type 0 interrupt occurs	OF, SF, ZF, AF, PF, CF undefined

Source
Reg8
Reg16
Mem8
Mem16

Assembly Language

Operation

DIV CL	AX is divided by CL; the unsigned quotient is in AL and the remainder is in AH
DIV CX	DX-AX is divided by CX; the unsigned quotient is in AX and the remainder is in DX
DIV NUMB	AX is divided by the contents of the data segment memory location NUMB; the unsigned quotient is in AX and the remainder is in DX

Division Instructions (DIV)

Mnemonic	Meaning	Format	Operation	Flags Affected
DIV	Division (unsigned)	DIV S	(1) $Q((AX)/(S8)) \rightarrow (AL)$ $R((AX)/(S8)) \rightarrow (AH)$ (2) $Q((DX,AX)/(S16)) \rightarrow (AX)$ $R((DX,AX)/(S16)) \rightarrow (DX)$ If Q is FF_{16} in case (1) or $FFFF_{16}$ in case (2), then type 0 interrupt occurs	OF, SF, ZF, AF, PF, CF undefined

Example:

```
MOV AX, 203      ; AX = 00CBh
MOV BL, 4
DIV BL           ; AL = 50 (32h), AH = 3
RET
```


Division Instructions (IDIV)

Mnemonic	Meaning	Format	Operation	Flags Affected
IDIV	Integer divide (signed)	IDIV S	(1) $Q((AX)/(S8)) \rightarrow (AL)$ $R((AX)/(S8)) \rightarrow (AH)$ (2) $Q((DX,AX)/(S16)) \rightarrow (AX)$ $R((DX,AX)/(S16)) \rightarrow (DX)$ If Q is positive and exceeds $7FFF_{16}$ or if Q is negative and becomes less than 8001_{16}, then type 0 interrupt occurs	OF, SF, ZF, AF, PF, CF undefined

Source
Reg8
Reg16
Mem8
Mem16

Assembly Language

Operation

IDIV BL	AX is divided by BL; the signed quotient is in AL and the remainder is in AH
IDIV SI	DX-AX is divided by SI; the signed quotient is in AX and the remainder is in DX

Division Instructions (IDIV)

Mnemonic	Meaning	Format	Operation	Flags Affected
IDIV	Integer divide (signed)	IDIV S	(1) $Q((AX)/(S8)) \rightarrow (AL)$ $R((AX)/(S8)) \rightarrow (AH)$ (2) $Q((DX,AX)/(S16)) \rightarrow (AX)$ $R((DX,AX)/(S16)) \rightarrow (DX)$ If Q is positive and exceeds $7FFF_{16}$ or if Q is negative and becomes less than 8001_{16} , then type 0 interrupt occurs type 0 interrupt occurs	OF, SF, ZF, AF, PF, CF undefined

Example:

```
MOV AX, -203 ; AX = 0FF35h
MOV BL, 4
IDIV BL      ; AL = -50 (0CEh), AH = -3 (0FDh)
RET
```

Division Instructions (AAD)

Mnemonic	Meaning	Format	Operation	Flags Affected
AAD	Adjust AX for division	AAD	$(AH) \cdot 10 + AL \rightarrow AL$ $00 \rightarrow AH$	SF, ZF, PF OF, AF, CF undefined

ASCII Adjust before Division.
Prepares two BCD values for division.

Algorithm:

- $AL = (AH * 10) + AL$
- $AH = 0$

Example:

```
MOV AX,00105h    ; AH = 01, AL = 05
AAD               ; AH = 00, AL = 0Fh (15)
RET
```


Division Instructions (AAD)

ASCII Adjust before Division.
Prepares two BCD values for division.

Algorithm:

- $AL = (AH * 10) + AL$
- $AH = 0$

AAD Instruction. Unlike all the other adjust instructions, the AAD instruction appears before a division. The AAD instruction requires that the AX register contain a 2-digit unpacked BCD number (not ASCII) before executing. After adjusting the AX register with AAD, it is divided by an unpacked BCD number to generate a single-digit result in AL with any remainder in AH.

Example illustrates how a 72 in unpacked BCD is divided by 9 to produce a quotient of 8. The 0702H loaded into the AX register is adjusted by the AAD instruction to 0048H. Notice that this converts a 2-digit unpacked BCD number into a binary number so it can be divided with the binary division instruction (DIV). The AAD instruction converts the unpacked BCD numbers between 00 and 99 into binary.

0000	B3 09	MOV	BL,009	;load divisor
0002	B8 0702	MOV	AX,00702H	;load dividend
0005	D5 0A	AAD		;adjust
0007	F6 F3	DIV	BL	

Division Instructions (CBW)

Mnemonic	Meaning	Format	Operation	Flags Affected
CBW	Convert byte to word	CBW	(MSB of AL) → (All bits of AH)	None

Convert byte into word.

Algorithm:

if high bit of AL = 1 then:

- AH = 255 (0FFh)

else

- AH = 0

Example:

```
MOV AX, 0      ; AH = 0, AL = 0
MOV AL, -5     ; AX = 000FBh (251)
CBW            ; AX = 0FFFBh (-5)
RET
```

Division Instructions (CWD)

Mnemonic	Meaning	Format	Operation	Flags Affected
CWD	Convert word to double word	CWD	(MSB of AX) → (All bits of DX)	None

Convert Word to Double word.

Algorithm:

if high bit of AX = 1 then:

- DX = 65535 (0FFFFh)

else

- DX = 0

Example:

```
MOV DX, 0      ; DX = 0
MOV AX, 0      ; AX = 0
MOV AX, -5     ; DX AX = 00000h:0FFFBh
CWD            ; DX AX = 0FFFFh:0FFFBh
RET
```


Logical AND Instruction (AND)

Mnemonic	Meaning	Format	Operation	Flags Affected
AND	Logical AND	AND D,S	$(S) \cdot (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined

The logical AND operation sets the destination bit according to the truth table shown below, depending on the values of the corresponding bits in the source and destination operands.

Example:

AL	BL	AND AL,BL AL
1010 1110	1111 0000	1010 0000
0110 0011	1001 1100	0000 0000
1100 0110	0000 0011	0000 0010
1111 0000	0000 1111	0000 0000

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

A	B	T
0	0	0
0	1	0
1	0	0
1	1	1



Logical AND Instruction (AND)

Mnemonic	Meaning	Format	Operation	Flags Affected
AND	Logical AND	AND D,S	$(S) \cdot (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined

An ASCII-coded number can be converted to BCD by using the AND instruction to mask off the leftmost four binary bit positions. This converts the ASCII 30H to 39H to 0-9. Example shows a short program that converts the ASCII contents of BX into BCD. The AND instruction in this example converts two digits from ASCII to BCD simultaneously.

EXAMPLE

```
0000 BB 3135      MOV  BX,3135H      ;load ASCII
0003 81 E3 0F0F    AND  BX,0F0FH      ;mask BX
```

x x x x	x x x x	Unknown number
• 0 0 0 0	1 1 1 1	Mask
0 0 0 0	x x x x	Result

Logical OR Instruction (OR)

Mnemonic	Meaning	Format	Operation	Flags Affected
OR	Logical Inclusive-OR	OR D,S	$(S) + (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined

The OR operation performs logical addition and is often called the Inclusive-OR function. The OR function generates a logic 1 output if any inputs are 1. A 0 appears at the output only when all inputs are 0. The truth table for the OR function appears below

Examples :

```
1.                ; AL = 1001 0011 = 93H
                  ; BL = 0111 0101 = 75H
OR BL, AL          ; OR byte in AL with byte in BL..
                  ; BL = 1111 0111 = F7H

2.                ; CX = 0110 1011 1001 1110
OR CX, 000F0H      ; CX = 0110 1011 1111 1110
```

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Truth table for the OR operation

source <i>bit</i>	destination <i>bit</i>	destination <i>bit</i>
0	0	0
0	1	1
1	0	1
1	1	1

Logical OR Instruction (OR)

Mnemonic	Meaning	Format	Operation	Flags Affected
OR	Logical Inclusive-OR	OR D,S	$(S) + (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined

Suppose that two BCD numbers are multiplied and adjusted with the AAM instruction. The result appears in AX as a 2-digit unpacked BCD number. Example below illustrates this multiplication and shows how to change the result into a 2-digit ASCII-coded number using the OR instruction.

```
MOV AL,005H  
MOV BL,007H  
MUL BL  
AAM  
OR AX, 03030H
```

Here, OR AX,03030H converts the 0305H found in AX to 3335H. The OR operation can be replaced with an ADD AX,03030H to obtain the same results.

Logical XOR Instruction (XOR)

Mnemonic	Meaning	Format	Operation	Flags Affected
XOR	Logical Exclusive-OR	XOR D,S	$(S) \oplus (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined

The Exclusive-OR instruction (XOR) differs from Inclusive-OR (OR). The difference is that a 1,1 condition of the OR function produces a 1, while the 1,1 condition of the Exclusive-OR operation produces a 0. The Exclusive-OR operation excludes this condition, while the Inclusive-OR includes it.

Examples :

```
1.          ; AL = 1010      1111 = AFH
          ; BL = 1111      0000 = F0H
XOR BL, AL  ; XOR byte in AL with byte in BL
          ; BL = 0101      1111 = 5FH
```

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

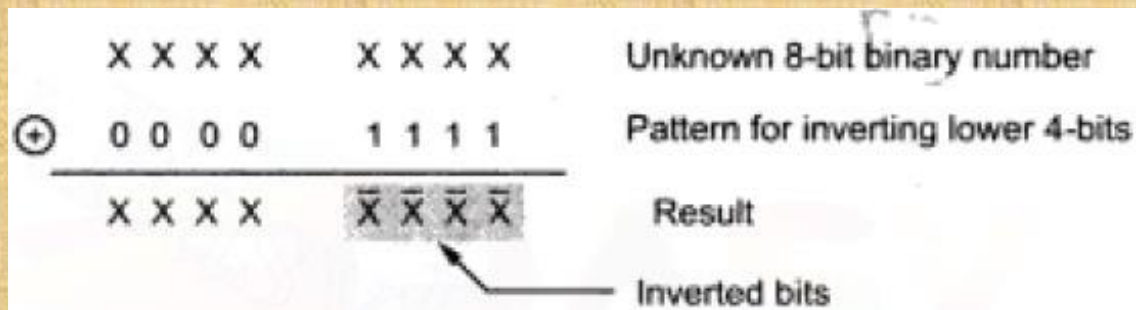
Truth table for the XOR operation

source <i>bit</i>	destination <i>bit</i>	destination <i>bit</i>
0	0	0
0	1	1
1	0	1
1	1	0

Logical XOR Instruction (XOR)

Mnemonic	Meaning	Format	Operation	Flags Affected
XOR	Logical Exclusive-OR	XOR D,S	$(S) \oplus (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined

- The XOR operation **inverts (complements)** some bits of a register or memory location.
- This instruction allows **part of a number** to be **inverted** or **complemented**. This illustrated in the figure below



Logical NOT Instruction (NOT)

Mnemonic	Meaning	Format	Operation	Flags Affected
NOT	Logical NOT	NOT D	$(D) \rightarrow (\bar{D})$	None

- The NOT instruction **inverts each bit** of a byte or a word.
- The destination can be a register or a memory location.
- NOT instruction affects no flags

Destination
Reg Memory

Examples :

```
NOT AL      ; AL = 0110 1100
             ; AL = 1001 0011
NOT CX      ; CX = 1010 1111 0010 0110
             ; CX = 0101 0000 1101 1001
```


BONUS TIME

Question: Calculate the content of AX register after executing the following instructions;

CLICK TO REVEAL

IF YOU ANSWER within THE FIRST TEN RESPONSES and your answer is correct you will get 5 points bonus for FINAL EXAM (you have 2min)

BONUS TIME

Question: Calculate the content of AX register after executing the following instructions;

```
MOV AX,033CCH  
AND AX,0FF00H  
XOR BX,BX  
NOT BX  
OR AX,BX
```

IF YOUR ANSWER IS TRUE WITHIN THE FIRST TEN RESPONSES you will get 5 points bonus for FINAL EXAM (you have 2min)

Shift Instructions (SAL/SHL)

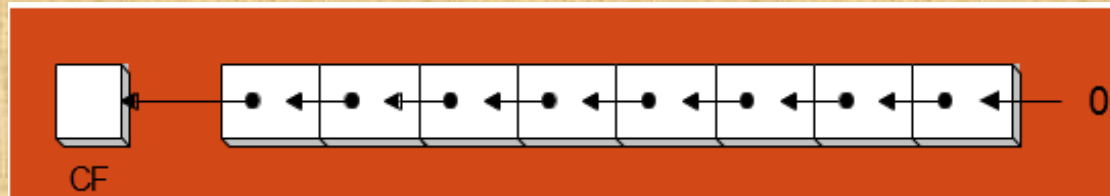
Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Mnemonic	Meaning	Format	Operation	Flags Affected
SAL/SHL	Shift arithmetic left/shift logical left	SAL/SHL D,Count	Shift the (D) left by the number of bit positions equal to Count and fill the vacated bits positions on the right with zeros	CF, PF, SF, ZF, OF AF undefined OF undefined if count ≠ 1

- SHL and SAL are two mnemonics for the same instruction.
- This instruction shifts each bit in the specified destination to the left and **0** is stored at LSB position. The MSB is shifted into the carry flag.
- The destination can be a byte or a word in a register or memory
- The number of shifts are indicated by count.
- If the number of shifts are greater than 1, then shift count must be loaded in CL register and CL must be placed in the count position of the instruction.

Shift Instructions (SAL/SHL)

Diagram bellow shows **SHL/SAL** instruction for byte operation



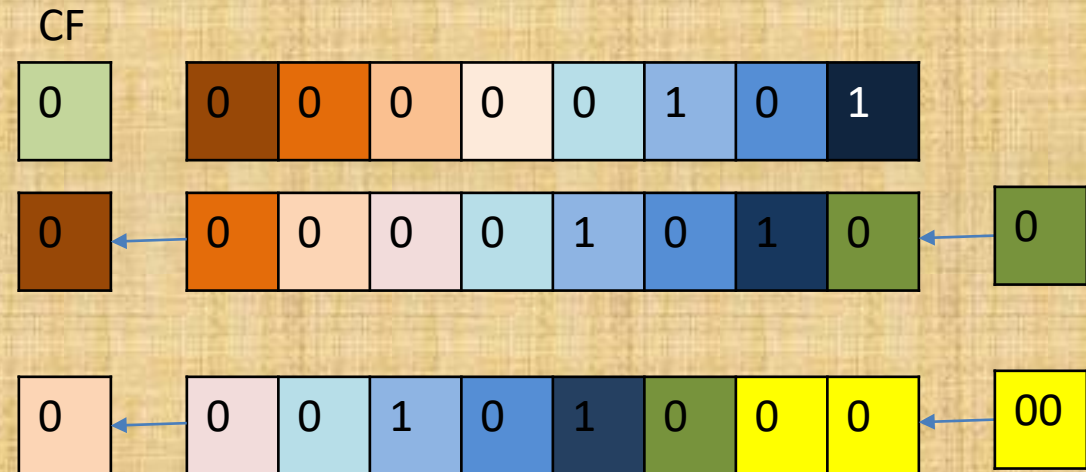
Example

MOV DL,005H

SHL DL,1

MOV CL, 002H

SAL DL,CL



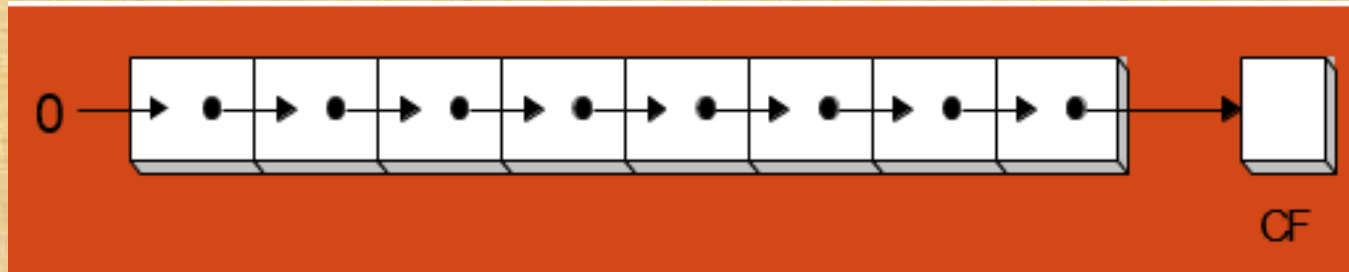
Shift Instructions (SHR)

Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Mnemonic	Meaning	Format	Operation	Flags Affected
SHR	Shift logical right	SHR D,Count	Shift the (D) right by the number of bit positions equal to Count and fill the vacated bit positions on the left with zeros	CF, PF, SF, ZF, OF AF undefined OF undefined if count $\neq 1$

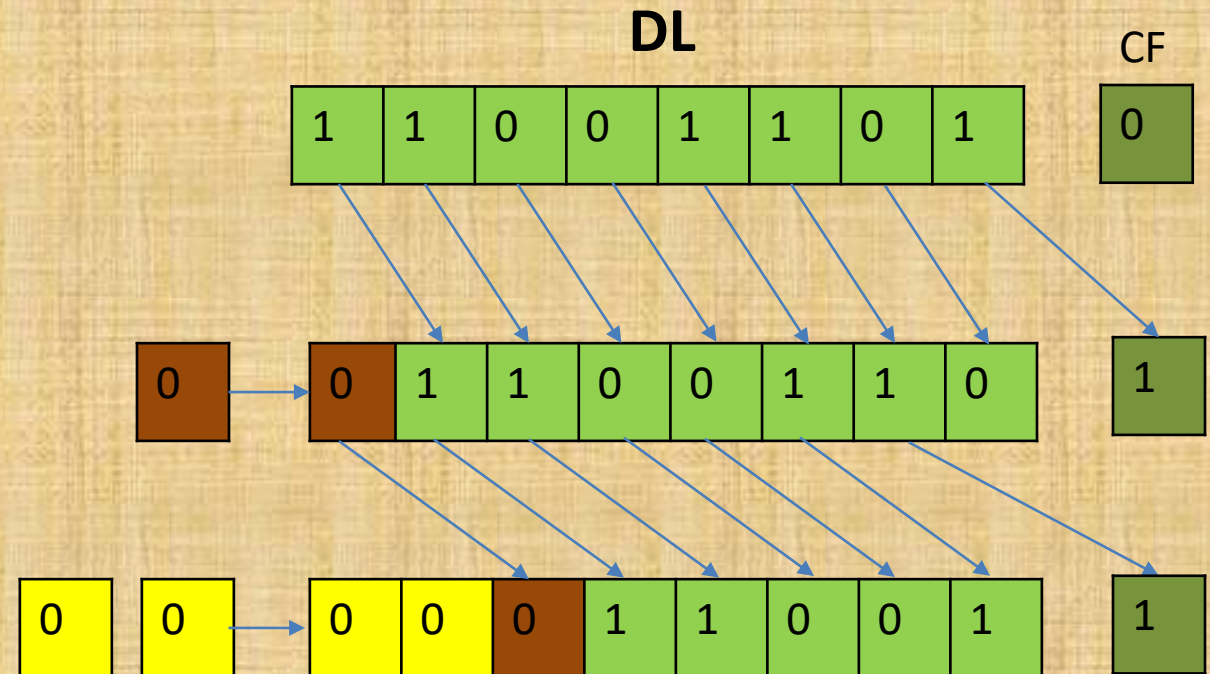
- This instruction shifts each bit in the specified destination to the right and 0 is stored at MSB position.
- The LSB is shifted into the carry flag.
- The destination can be a byte or a word in a register or in a memory location.
- The number of shifts are indicated by count.
- If the number of shifts are greater than 1, then shift count must be loaded in CL register and CL must be placed in the count position of the instruction.

Shift Instructions (SHR)



Example

```
MOV DL,0CDH  
SHR DL,1  
MOV CL, 002H  
SHR DL,CL
```



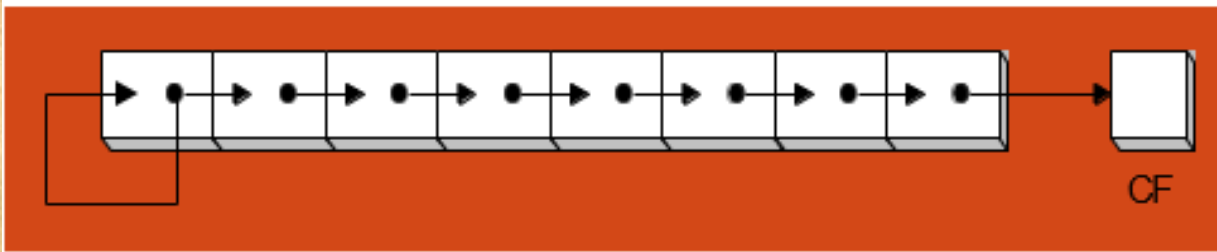
Shift Instructions (SAR)

Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Mnemonic	Meaning	Format	Operation	Flags Affected
SAR	Shift arithmetic right	SAR D,Count	Shift the (D) right by the number of bit positions equal to Count and fill the vacated bit positions on the left with the original most significant bit	OF, SF, ZF, PF, CF AF undefined

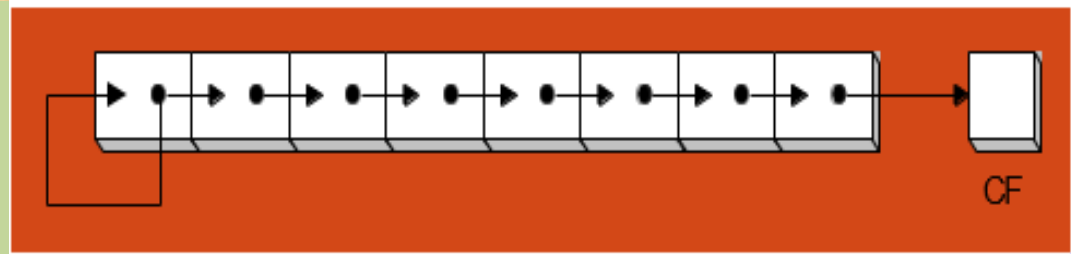
- This instruction shifts each bit in the specified destination some number of bit positions to the right.
- As a bit is shifted out of the MSB position, a copy of the old MSB is put in the MSB position.
- The LSB will be shifted into CF
- In case of multiple shifts , CF will contain the bit most recently shifted in from the LSB.
- Bits Shifted into CF previously will be lost.

Shift Instructions (SAR)



- An arithmetic shift preserves the number's sign.
- The number of shifts are indicated by count.
- If number of shifts required is one, you can place one (1) in the count position.
- If the number of shifts are greater than 1, then shift count must be loaded in CL register and CL must be placed in the count position of the instruction.

Shift Instructions (SAR)



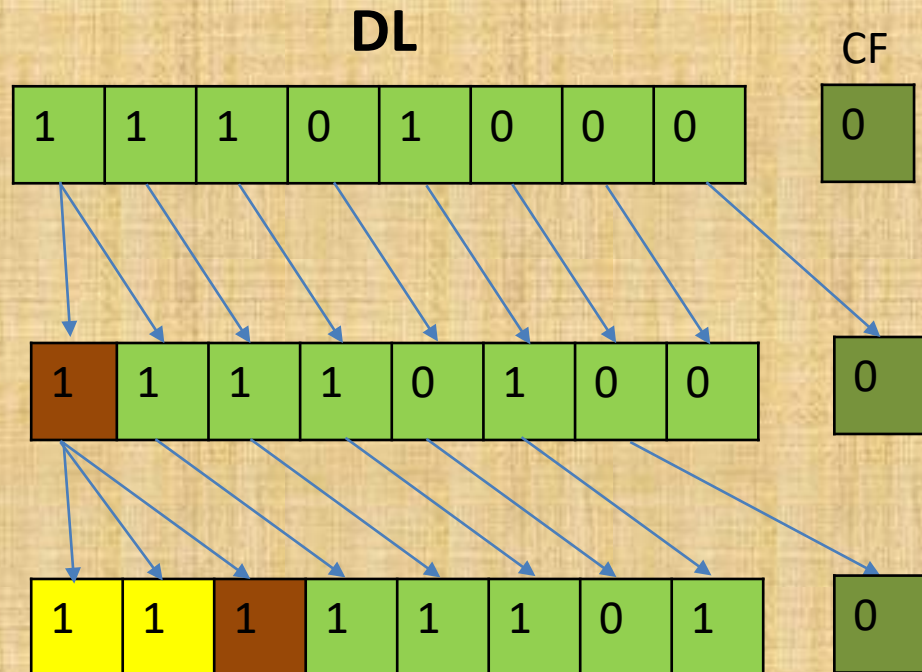
Example

MOV DL,0E8H

SAR DL,1

MOV CL, 002H

SAR DL,CL

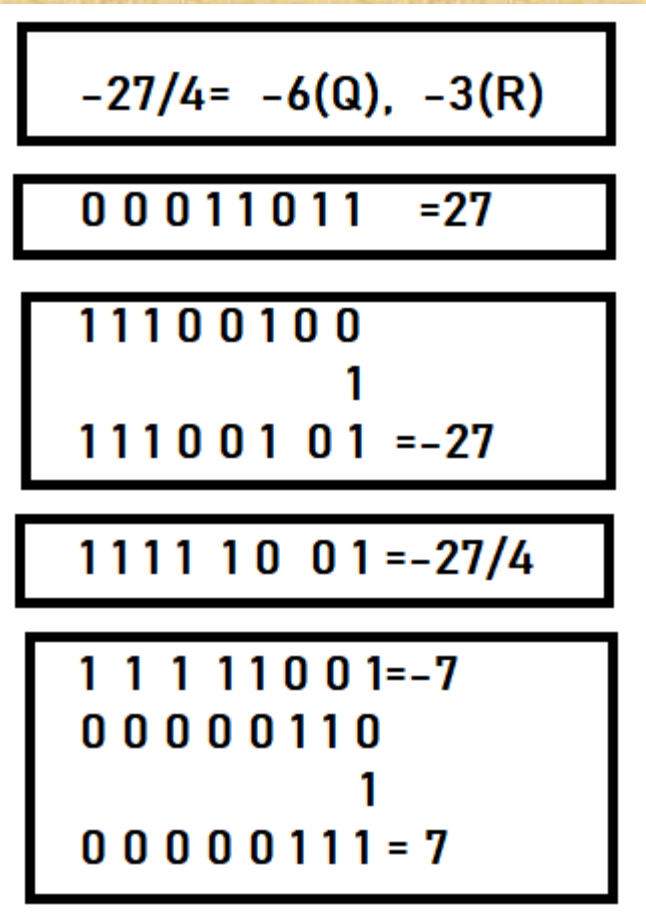


DL

(E8)h = (232)u = (-24)s

(F4)h = (244)u = (-12)s

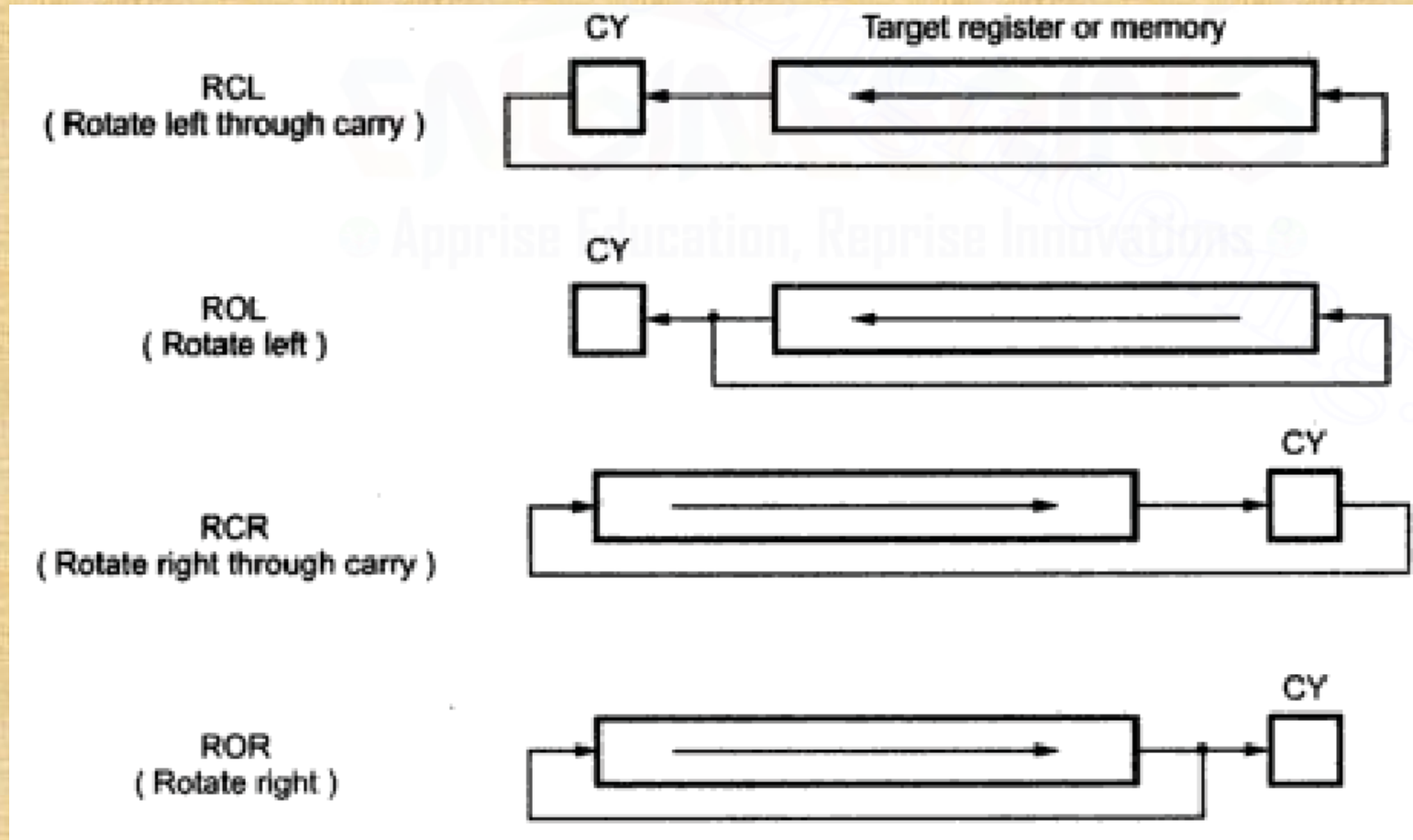
(FD)h = (253)u = (-3)s



Do not use it for division of negative numbers

Rotate Instructions (RCL,ROL,RCR,ROR)

These instructions **position or move** binary data by **rotating** the information in a register or memory location, either from one end to another or through the carry flag



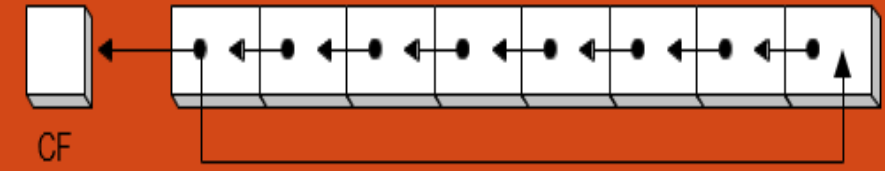
Rotate Instructions (ROL)

Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Mnemonic	Meaning	Format	Operation	Flags Affected
ROL	Rotate left	ROL D,Count	Rotate the (D) left by the number of bit positions equal to Count. Each bit shifted out from the leftmost bit goes back into the rightmost bit position.	CF OF undefined if count $\neq 1$

- This instruction rotates all bits in a specified byte or word to the left some number of bit positions.
- MSB is placed as new LSB and a new CF.
- The destination can be a byte or a word in a register or in a memory location.
- The number of shifts are indicated by count.
- If the number of shifts required is one, you can place one (1) in the count position.
- If the number of shifts are greater than 1, then shift count must be loaded in CL register and CL must be placed in the count position of the instruction

Rotate Instructions (ROL)



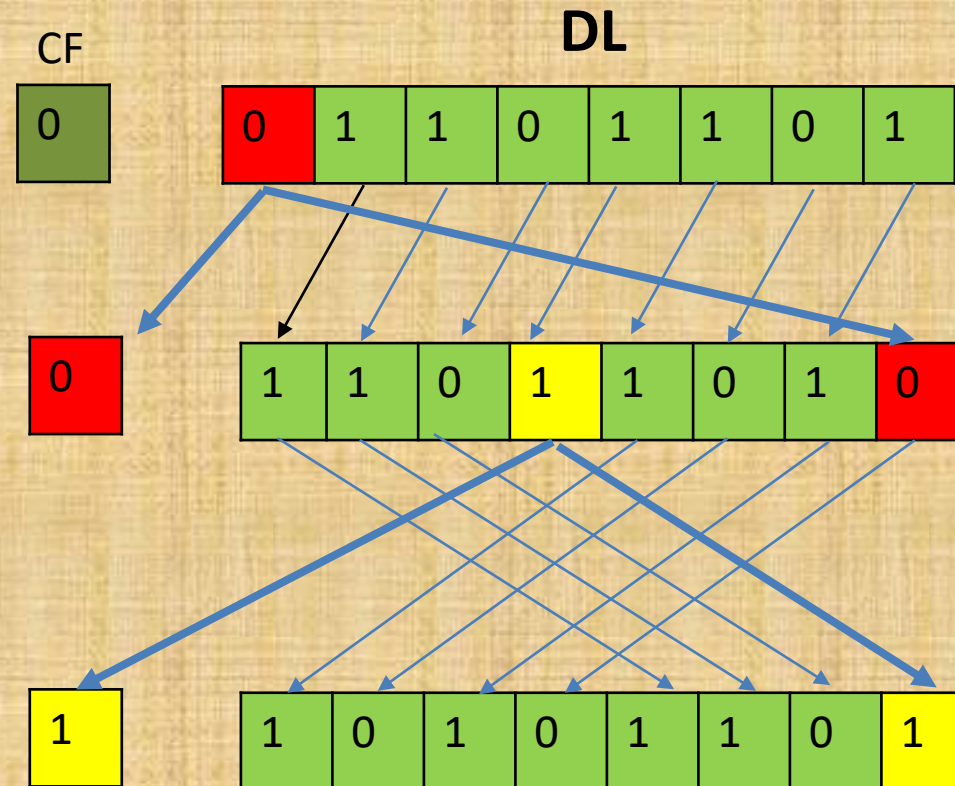
Example

```
MOV DL,001H
```

```
ROL DL,1
```

```
MOV CL, 004H
```

```
ROL DL,CL
```



- ROL (rotate) shifts each bit to the left

- The highest bit is copied into both the Carry flag and into the lowest bit

- **No bits are lost**

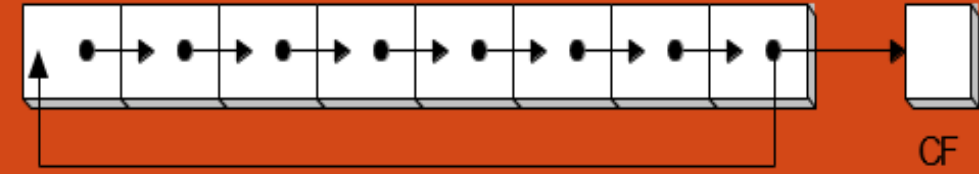
Rotate Instructions (ROR)

Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Mnemonic	Meaning	Format	Operation	Flags Affected
ROR	Rotate right	ROR D.Count	Rotate the (D) right by the number of bit positions equal to Count. Each bit shifted out from the rightmost bit goes into the leftmost bit position.	CF OF undefined if count $\neq 1$

- This instruction rotates all bits in a specified byte or word to the right some number of bit positions.
- LSB is placed as new MSB and a new CF.
- The destination can be a byte or a word in a register or in a memory location.
- The number of shifts are indicated by count.
- If number of shifts required is one, you can place one (1) in the count position.
- If the number of shifts are greater than 1, then shift count must be loaded in CL register and CL must be placed in the count position of the instruction

Rotate Instructions (ROR)



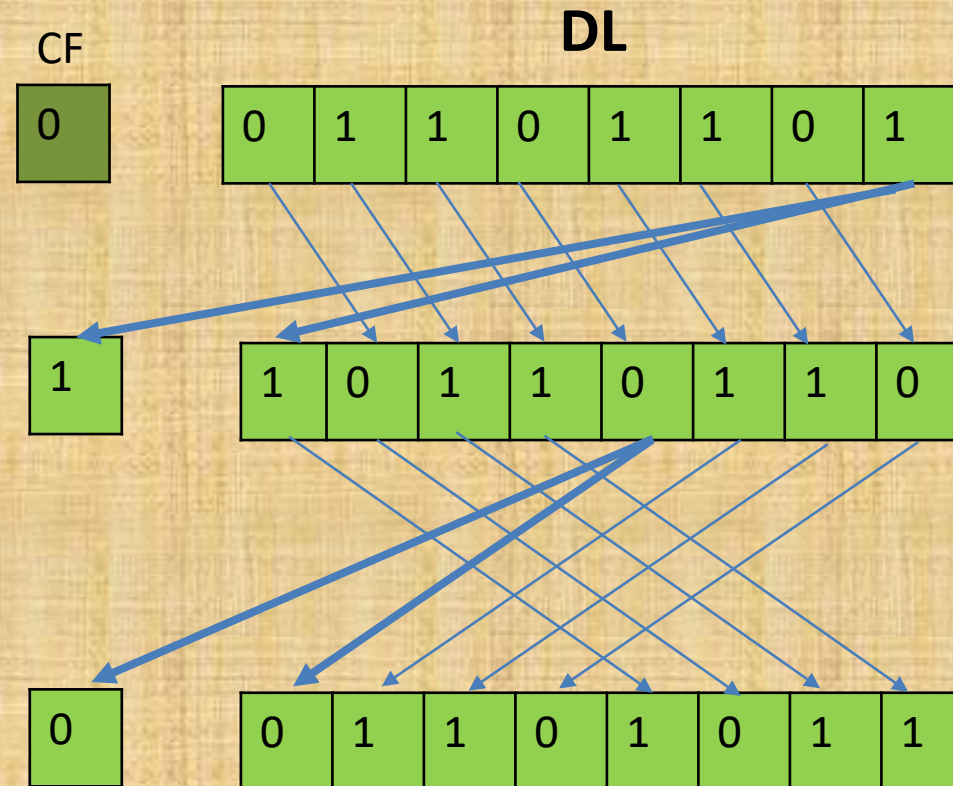
Example

```
MOV DL,001H
```

```
ROL DL,1
```

```
MOV CL, 004H
```

```
ROL DL,CL
```



- ROR (rotate) shifts each bit to the right

- The lowest bit is copied into both the Carry flag and into the highest bit

- **No bits are lost**

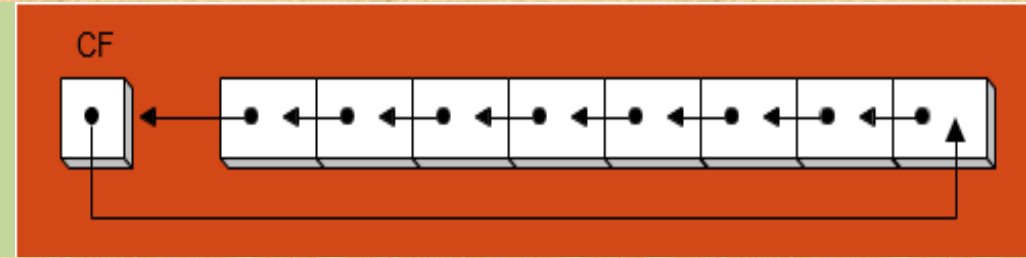
Rotate Instructions (RCL)

Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Mnemonic	Meaning	Format	Operation	Flags Affected
RCL	Rotate left through carry	RCL D,Count	Same as ROL except carry is attached to (D) for rotation.	CF OF undefined if count $\neq 1$

- This instruction rotates all bits in a specified byte or word some number of bit positions to the left along with the carry flag .
- MSB is placed as new Carry and previous Carry is placed as a new LSB. The destination can be a byte or a word in a register or in a memory location.
- The number of shifts are indicated by count.
- If number of shifts required is one, you can place one (1) in the count position.
- If the number of shifts are greater than 1, then shift count must be loaded in CL register and CL must be placed in the count position of the instruction

Rotate Instructions (RCL)



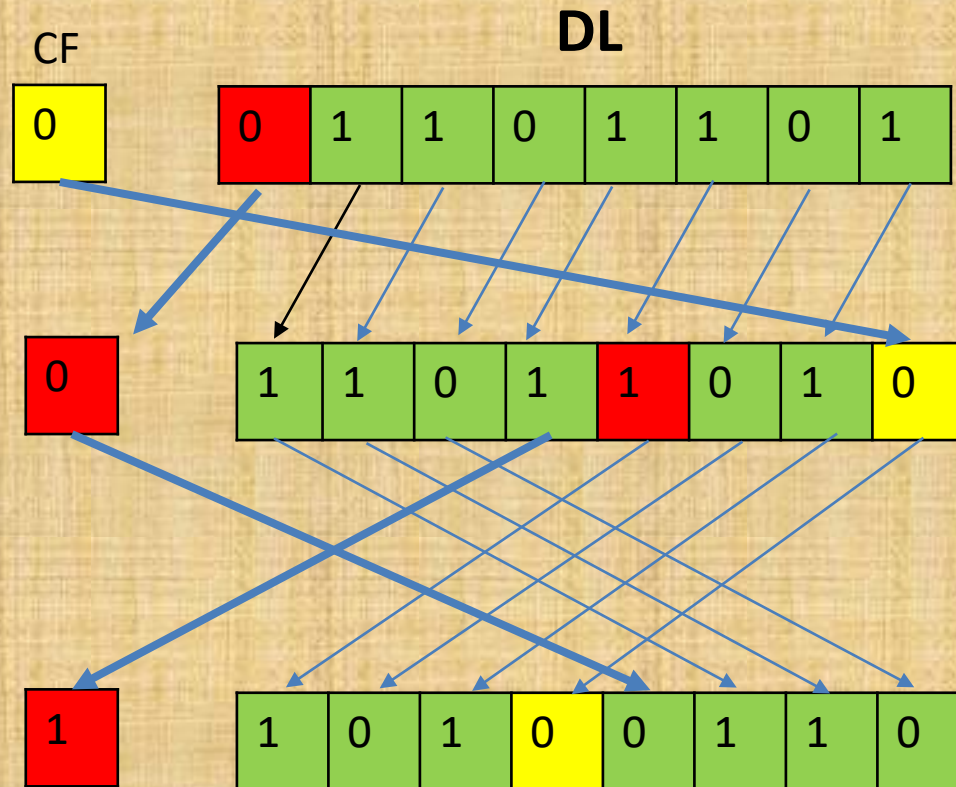
Example

MOV DL,001H

RCL DL,1

MOV CL, 004H

RCL DL,CL



- RCL (rotate left through carry) shifts each bit to the left
- Copies the Carry flag to the least significant bit
- Copies the most significant bit to the Carry flag

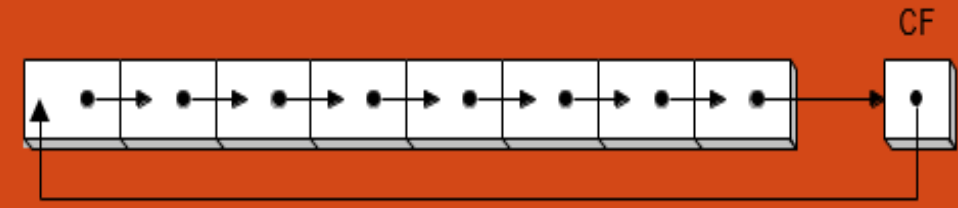
Rotate Instructions (RCR)

Destination	Count
Register	1
Register	CL
Memory	1
Memory	CL

Mnemonic	Meaning	Format	Operation	Flags Affected
RCR	Rotate right through carry	RCR D, Count	Same as ROR except carry is attached to (D) for rotation.	CF OF undefined if count $\neq 1$

- This instruction rotates all bits in a specified byte or word some number of bit positions to the right along with the carry flag .
- LSB is placed as new Carry and previous Carry is placed as a new MSB.
- The destination can be a byte or a word in a register or in a memory location.
- The number of shifts are indicated by count.
- If number of shifts required is one, you can place one (1) in the count position.
- If the number of shifts are greater than 1, then shift count must be loaded in CL register and CL must be placed in the count position of the instruction

Rotate Instructions (RCR)



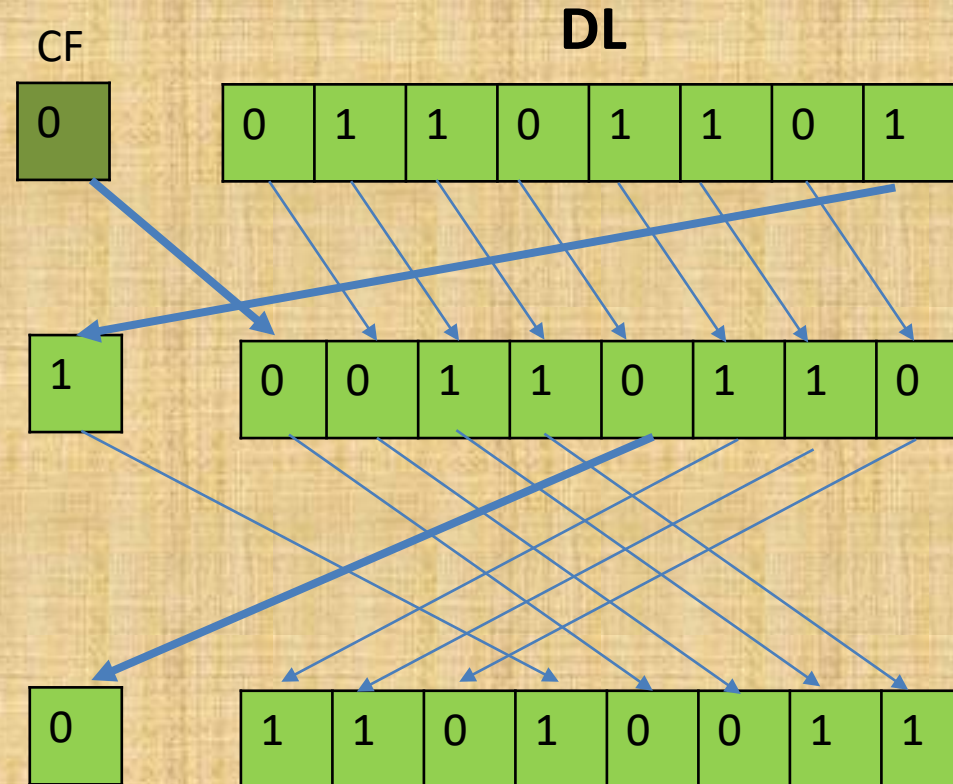
Example

MOV DL,001H

RCR DL,1

MOV CL, 004H

RCR DL,CL



RCR (rotate right through carry) shifts each bit to the right

- Copies the Carry flag to the most significant bit
- Copies the least significant bit to the Carry flag

$$7(AX) - 5(BX) - \frac{1}{8}(BX) \rightarrow (AX)$$

Implement the above operation using shift and arithmetic instructions(Multiplication and Division instructions are not allowed)

MOV CL,003H	Load CL with 3 to get ready for multiplication by 8
MOV DX,AX	Make a copy of AX into DX (we will need this later on)
SHL AX,CL	Shifting AX 3 times to left is equivalent to multiplying it by 8
SUB AX,DX	$8(AX) - (AX) = 7(AX)$ is stored into AX
MOV DX,BX	Make a copy of BX into DX (we will need this later on)
MOV CL,002H	Load CL with 2 to get ready for multiplication by 4
SHL BX,CL	Shifting BX 2 times to left is equivalent to multiplying it by 4
ADD BX,DX	$4(BX) + (BX) = 5(BX)$ is stored into BX
MOV CL,003H	Load CL with 3 to get ready for division by 8
SHR DX,CL	Shifting DX 3 times to right is equivalent to dividing it by 8
SUB AX,BX	$7(AX) - 5(BX) \rightarrow (AX)$
SUB AX,DX	$7(AX) - 5(BX) - \frac{1}{8}(BX) \rightarrow (AX)$

FLAG Control Instructions

Mnemonic	Meaning	Operation	-Flags Affected-
LAHF	Load AH from flags	$(AH) \leftarrow (\text{Flags})$	None
SAHF	Store AH into flags	$(\text{Flags}) \leftarrow (AH)$	SF, ZF, AF, PF, CF
CLC	Clear carry flag	$(CF) \leftarrow 0$	CF
STC	Set carry flag	$(CF) \leftarrow 1$	CF
CMC	Complement carry flag	$(CF) \leftarrow (\overline{CF})$	CF
CLI	Clear interrupt flag	$(IF) \leftarrow 0$	IF
STI	Set interrupt flag	$(IF) \leftarrow 1$	IF
CLD	Clear direction flag	$(DF) \leftarrow 0$	DF
STD	Set direction flag	$(DF) \leftarrow 1$	DF

AH register

S Z A P I C

Compare Instruction (CMP)

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Mnemonic	Meaning	Format	Operation	Flags Affected
CMP	Compare	CMP D,S	(D) – (S) is used in setting or resetting the flags	CF, AF, OF, PF, SF, ZF

This instruction compares the contents of source operand with the contents of destination operands.

- For comparison, it **subtracts the source operand from the destination operand** but **does not store the result anywhere**.
- The source may be immediate data, memory location or register.
- The destination may be memory location or register.
- The source and destination both can't be memory location.

Compare Instruction (CMP)

RECALL

Sign Flag After any operation if the MSB (B(7)) of the result is 1, it indicates the number is negative and the sign flag becomes set, i.e. 1. If the MSB is 0, it indicates the number is positive and the sign flag becomes reset i.e. 0.

Zero Flag After any arithmetical or logical operation if the result is 0 (00)H, the zero flag becomes set i.e. 1, otherwise it becomes reset i.e. 0.

Auxiliary Carry Flag If after any arithmetic or logical operation B3 generates any carry and passes on to B4 this flag becomes set i.e. 1, otherwise it becomes reset i.e. 0.

Carry Flag (CY) – Carry is generated when performing n bit operations and the result is more than n bits, then this flag becomes set i.e. 1, otherwise it becomes reset i.e. 0.

Parity Flag (P) – If after any arithmetic or logical operation the result has even parity, an even number of 1 bits, the parity register becomes set i.e. 1, otherwise it becomes reset i.e. 0.

Overflow Flag This flag will be set (1) if the result of a signed operation is too large to fit in the number of bits available to represent it, otherwise reset (0). After any operation, if B6 generates any carry and passes to B7

OR if B6 does not generate carry but B7 generates, overflow flag becomes set, i.e., 1.

If B6 and B7 both generate carry or both do not generate any carry, then overflow flag becomes reset, i.e., 0.

Compare Instruction (CMP)

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Example: Calculate the flags S,Z,A,C,P,O after executing the last instruction

MOV AL,099H
MOV BL, 01BH
CMP AL,BL

S:

Z:

A:

C:

P:

O:

1 0 0 1 1 0 0 1 AL -103d
0 0 0 0 1 0 1 1 BL +27d

$-103 - (+27) = -103 + (-27)$

0 0 0 1 1 0 1 1 +27d

1 1 1 0 0 1 0 0 2's complement

1

1 1 1 0 0 1 0 1 -27d

1 0 0 1 1 0 0 1 AL -103d

1 1 1 0 0 1 0 1 -27d

Jump Instruction (JMP)

Mnemonic	Meaning	Format	Operation	Flags Affected
JMP	Unconditional jump	JMP Operand	Jump is initiated to the address specified by Operand	None

Operand
Short-label
Near-label
Far-label
Memptr16
Regptr16
Memptr32

This instruction causes an instruction is to be fetched **from the location specified in the JMP instruction** rather than from the next location after the JMP instruction.

- **Short JUMP**

Allows jumps or branches to memory locations within +127 and -128 bytes from the address of the instruction following the jump.

- **NEAR JUMP**

Allows a jump or branch to memory locations within ± 32 Kbytes (or anywhere in the current code segment) from the address of the instruction following the jump. Only IP is changed, the contents of the CS remains the same.

- **Far JUMP**

Allows a jump or branch to any memory location within the real memory system. Destination location is from a different segment. Both IP and CS changed.

The **SHORT** and **NEAR** jumps are called **intra-segment** jumps, and the **FAR** jumps are called **inter-segment** jumps.

- A **positive displacement** usually means jump is 'ahead' in the program, and a **negative displacement** usually means jump is 'backward' in the program.

FIGURE A short jump to four memory locations beyond the address of the next instruction

JMP SHORT NEXT

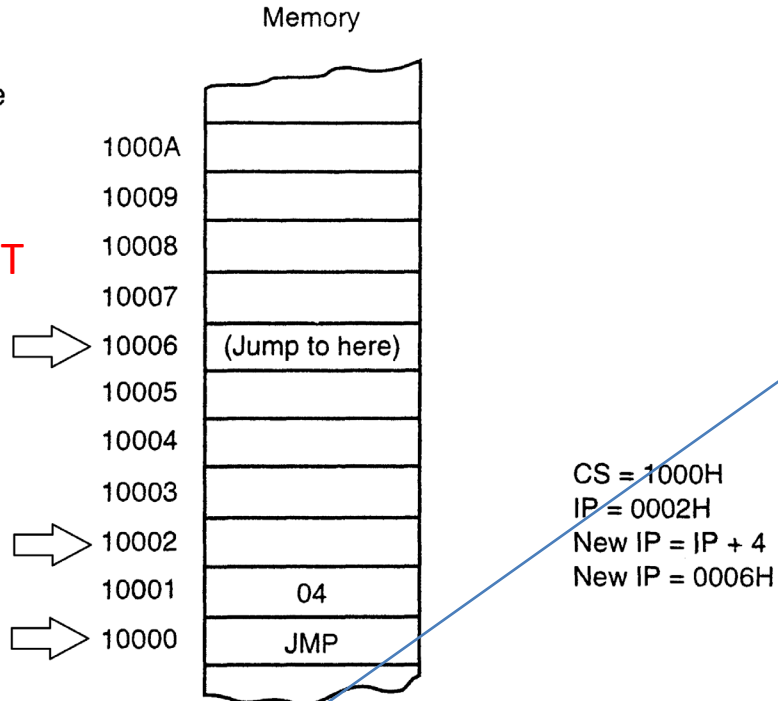


FIGURE A near jump that adds the displacement (0002H) to the contents of IP

JMP NEAR NEXT

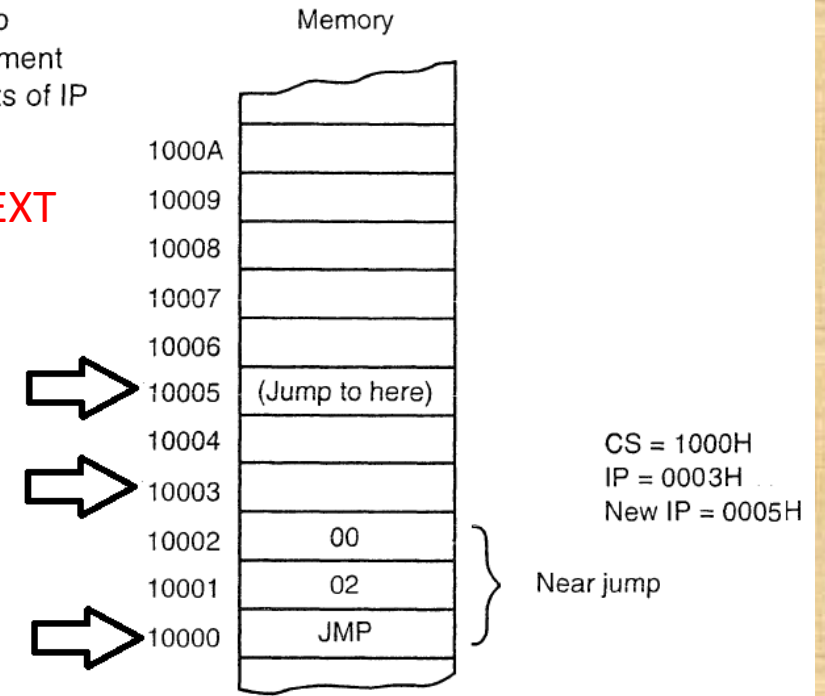
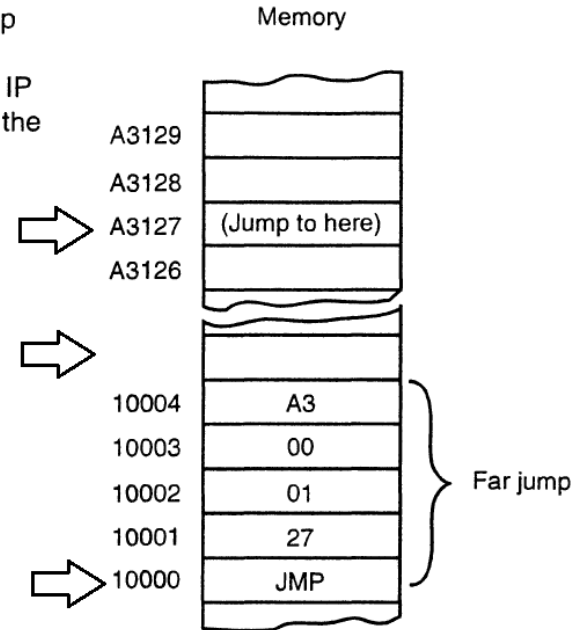


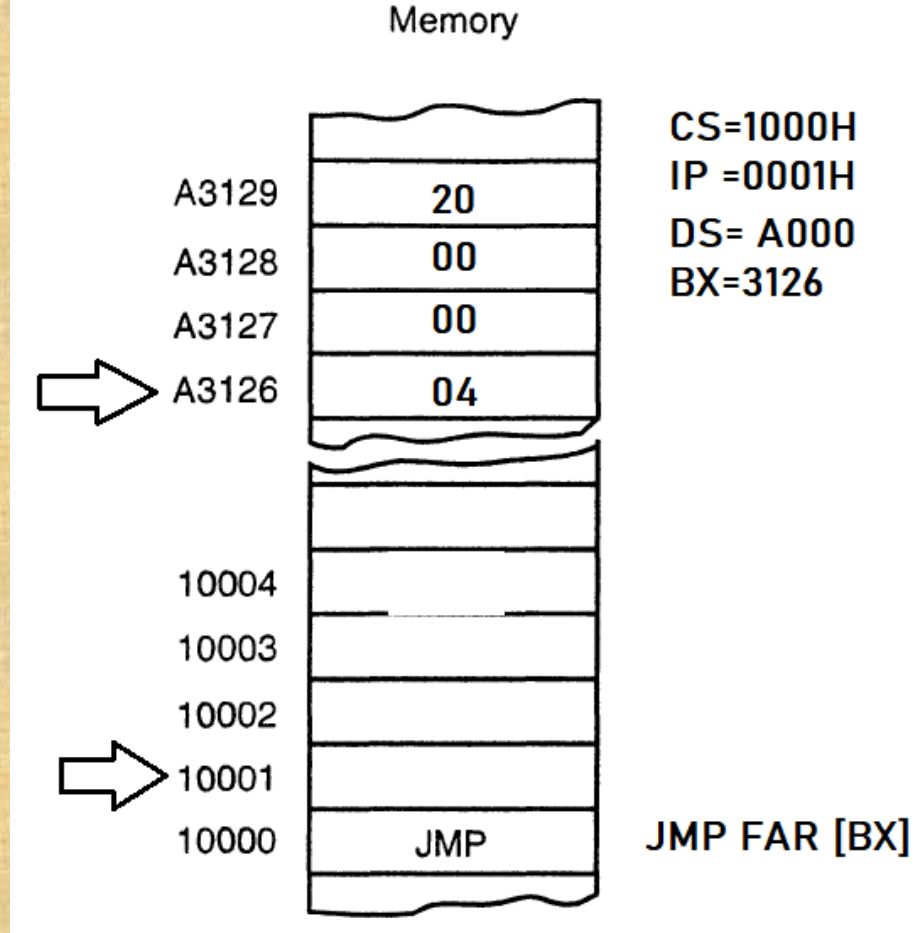
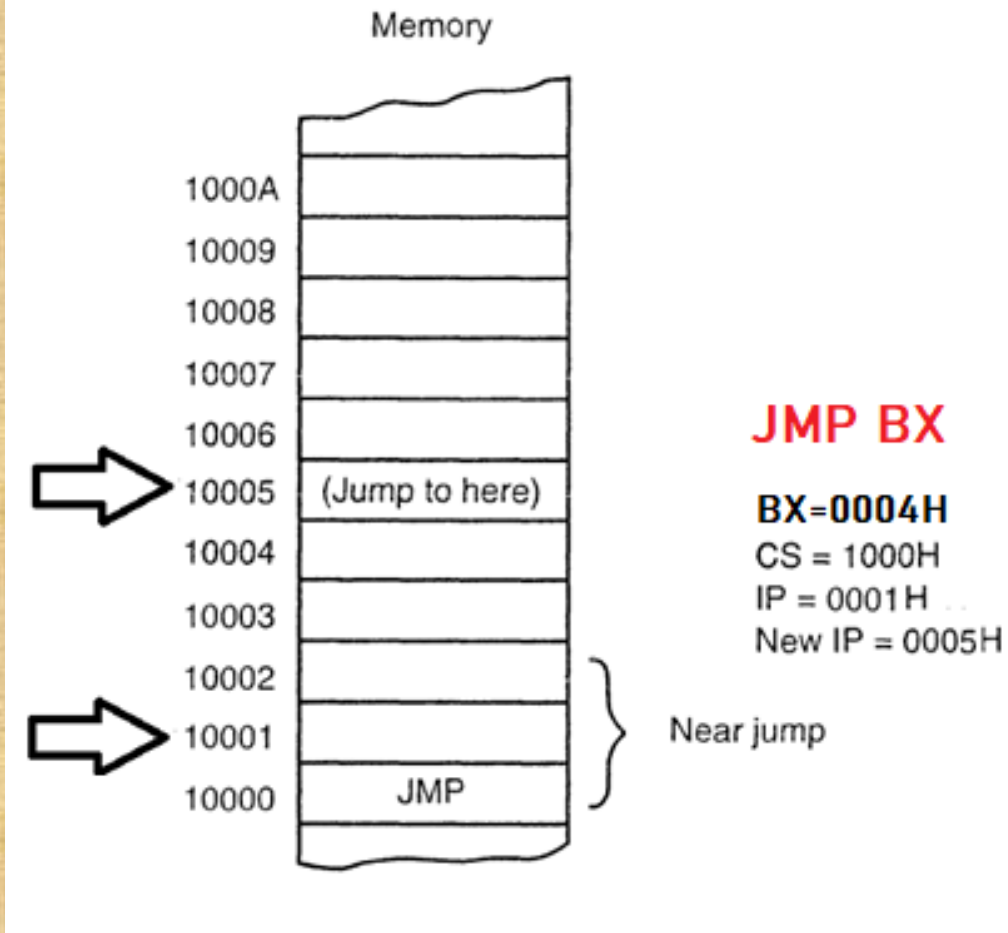
FIGURE A far jump instruction replaces the contents of both CS and IP with four bytes following the opcode

JMP FAR NEXT



Operand
Short-label
Near-label
Far-label
Memptr16
Regptr16
Memptr32

Jump Instruction (JMP)



Operand
Short-label
Near-label
Far-label
Memptr16
Regptr16
Memptr32

Conditional Jump Instructions (Jcc)

Mnemonic	Meaning	Format	Operation	Flags Affected
Jcc	Conditional jump	Jcc Operand	If the specific condition cc is true, the jump to the address specified by the Operand is initiated; otherwise, the next instruction is	None

Operand
Short-label
Near-label
Far-label
Memptr16
Regptr16
Memptr32

Assembly Language	Condition Tested	Operation
JA	$Z = 0$ and $C = 0$	Jump if above
JAE	$C = 0$	Jump if above or equal
JB	$C = 1$	Jump if below
JBE	$Z = 1$ or $C = 1$	Jump if below or equal
JC	$C = 1$	Jump if carry set
JE or JZ	$Z = 1$	Jump if equal or jump if zero
JG	$Z = 0$ and $S = 0$	Jump if greater than
JGE	$S = 0$	Jump if greater than or equal
JL	$S \neq 0$	Jump if less than
JLE	$Z = 1$ or $S \neq 0$	Jump if less than or equal
JNC	$C = 0$	Jump if no carry
JNE or JNZ	$Z = 0$	Jump if not equal or jump if not zero
JNO	$O = 0$	Jump if no overflow
JNS	$S = 0$	Jump if no sign
JNP or JPO	$P = 0$	Jump if no parity or jump if parity odd
JO	$O = 1$	Jump if overflow set
JP or JPE	$P = 1$	Jump if parity set or jump if parity even
JS	$S = 1$	Jump if sign is set
JCXZ	$CX = 0$	Jump if CX is zero

Less/Greater Signed

Above/Below Unsigned

CALL instruction (CALL)

Mnemonic	Meaning	Format	Operation	Flags Affected
CALL	Subroutine call	CALL operand	Execution continues from the address of the subroutine specified by the operand. Information required to return back to the main program such as IP and CS are saved on the stack.	None

Operand
Near-proc Far-proc Memptr16 Regptr16 Memptr32

It is used to call a procedure and save their return address to stack

Types of Procedure

- depends on where the procedure is stored in the main memory.

Near Procedure & Far Procedure

If it is in the same code segment where the main program is stored it is referred to as **near procedure**, otherwise it is called **far procedure**.

Near Procedure (i.e., ± 32 KB displacement)

For **near procedures**, **CALL** instruction pushes only the **IP** register contents on the stack, since **CS register** contents **remains unchanged** for main program and procedure

Far Procedure (any where outside the current segment)

But for **far procedures**, **CALL** instruction pushes **IP** and **CS** on the stack.

CALL instruction (CALL)

Far CALL

When a **far call** is being executed;

$((SP)-1) \leftarrow \text{HighCS}$

$((SP)-2) \leftarrow \text{LowCS}$

$((SP)-3) \leftarrow \text{HighOffset(Next Instruction)}$

$((SP)-4) \leftarrow \text{LowOffset(Next Instruction)}$

$(SP) \leftarrow SP-4$

Finally, **CS** will be get loaded with the **base of the destination segment** which contains the procedure and **IP** will be get loaded with **offset of the first instruction of the procedure** in the destination segment.

Near CALL is a call to a **procedure** which is in the **same code segment** as the **CALL** instruction.

When a **near call** is being executed,
 $((SP)-1) \leftarrow \text{HighOffset(Next Instruction)}$

$((SP)-2) \leftarrow \text{LowOffset(Next Instruction)}$
 $(SP) \leftarrow SP-2$

the **address (offset address on IP) of the next instruction** after the **CALL** instruction, is copied onto the stack. Then **SP** is **decremented by 2**. After that **IP** is get loaded with the offset of the first instruction of the procedure in the same segment.

Return instruction (CALL)

Mnemonic	Meaning	Format	Operation	Flags Affected
RET	Return	RET	Return to the main program by restoring IP (and CS for far-proc). If Operand is present, it is added to the contents of SP.	None

RET: Return from the procedure/subroutine.

- will return execution from a procedure to the next instruction after the CALL instruction in the calling program.
- If the procedure is a near procedure, then the return will be done by replacing the IP register with a word from the top of the stack. The SP will then be incremented by two.
- If the procedure is a far procedure; then IP register will be loaded with the word at the top of the stack. The SP will then be incremented by two. The CS register is then get loaded with a word form the new top of the stack. Then SP is again incremented by 2.
- No flags are affected with RET instruction.

Return instruction (CALL)

